

化学物質管理システム 詳細機能仕様書

項目	内容
版	1.0 (2026-07-03)
目的	本書のみを読んで、 同一の振る舞いのアプリケーションを再現実装 できる粒度で機能を定義する
範囲	機能詳細 (API入出力・検証・アルゴリズム・画面挙動)。非機能・体制・見積は対象外 (発注用仕様書を参照)
記法	「エラー(コード, HTTPステータス): メッセージ」形式で例外を厳密に列挙する。日本語メッセージは画面表示文言と一致させること

この文書は 2026-07 時点の記述です。いま動いている仕様は、アプリ内の「ユーザーマニュアル」(/spec) と docs/steps/、設計判断は docs/decisions/ にあります。DB は prisma/schema.prisma、判定は docs/judgment-engine.md が正です。2026-08 以降に足した機能の一覧は、本書の末尾 (または docs/steps/) をご覧ください。

0. 共通規約 (全機能に適用)

0.1 API共通

- すべてのAPIは JSON を返す。成功時は 2xx、失敗時は下記の統一エラー形式。
- エラー形式:** { "error": { "code": string, "message": string, "details"? : any } }
- 代表エラー (全APIで発生しうる) :
 - unauthorized, 401, "ログインが必要です" ... 未認証または無効化ユーザー
 - forbidden, 403, 権限不足 (メッセージは機能ごと)
 - invalid_json, 400, "リクエストボディがJSONではありません"
 - validation_error, 400, "入力内容に誤りがあります" (details に Zod の flatten または {errors:[...]})
 - not_found, 404, 対象なし (メッセージは機能ごと)
- 認可ヘルパ** (すべての変更系APIの先頭で必ず呼ぶ) :
 - requireUser(): 認証必須。未認証→401。
 - requireEditor(): 認証 + 編集権限 (後述 0.4) 必須。権限なし→ forbidden 403 "編集権限がありません (参照・判定実行・ダウンロードのみ可能です)"。
 - requireAdmin(): システム管理者のみ。それ以外→ forbidden 403 "システム管理者のみ実行できます"。
- 一覧の既定:** page = 1、pageSize = 50 (最大200)。応答は { items, total, page, pageSize }。
- 認可はサーバ側で必ず判定する。画面のボタン非表示は補助にすぎない。

0.2 文字列正規化（突合の基礎）

- `normalize(s) = s.trim().toUpperCase()`。
- コード類（物質コード・製品コード・法律コード・区分コード・CAS番号）は、一意判定・検索・突合をすべて**正規化値**（`*_normalized` 列）で行う。表示は原文を保持。
- これによりDBの照合順序差（大文字小文字の扱い）に依存しない。

0.3 数値

- 含有率・閾値・係数・割合はすべて十進数（Decimal）で計算。浮動小数点を使わない。
- 入力形式（文字列で受ける）：正規表現 `^\d{1,3}(\.\d{1,6})?$`（整数1～3桁、小数6桁まで）。
- 含有率・割合の値域： $0 < x \leq 100$ 。閾値： $0 \leq x \leq 100$ （上限のみ100制約、下限は正規表現のみ）。
- 保存精度：統合含有率・寄与・算定値は小数12桁で保存。表示は各画面指定桁。

0.4 論理削除・参照制約

- 全マスタは論理削除（`deleted_at` に日時をセット。既定NULL＝有効）。一覧・検索・詳細は `deleted_at IS NULL` のみ対象。
- 参照されているデータの削除は「1回目＝警告(409)」 「`?force=true` 付き＝強制削除」の2段階。
- 監査ログ（AuditLog）に全変更を記録： `{ entity, entityId, action(create/update/delete/login/determine/import/export), actorId, at, diff }`。書込失敗は業務を止めない（ログのみ）。

1. 認証・セッション・認可

1.1 認証（Supabase Auth）

- メール＋パスワード。ログイン画面 `/login` からクライアントSDKで `signInWithPassword`。
 - 失敗時は理由を問わず画面に「メールアドレスまたはパスワードが正しくありません」（アカウント存在を推測させない）。
- セッションはCookieで保持。サーバ側は各リクエストで `supabase.auth.getUser()` によりJWTを検証する。

1.2 アプリユーザーの自動生成（JITプロビジョニング）

`getSessionUser()` の手順:

1. Supabaseから現在ユーザーを取得。取得不可→null（未認証）。
2. `users` を `supabaseUserId` で検索。
 - 存在し `deletedAt` が非NULL または `activeFlag=false` →null（拒否）。
 - 存在し有効→そのユーザー。

3. 存在しない場合、新規作成: - `users` が0件なら `role=SYSTEM_ADMIN`, `canEdit=true` (最初のユーザーは管理者)。 - それ以外は `role=NON_PRIVILEGED`, `canEdit=false`。 - `displayName` にはメールアドレスを設定。

1.3 権限判定関数

- `canEdit(user) = role ∈ {SYSTEM_ADMIN, PRIVILEGED}` または `canEdit=true`。
- `isPrivileged(user) = role ∈ {SYSTEM_ADMIN, PRIVILEGED}`。

1.4 画面保護 (ミドルウェア)

- 対象: `/api` 及び静的アセットを除く全ページ。
- 未認証で保護ページ→ `/login` へ 307 リダイレクト。認証済みで `/login` → `/` へリダイレクト。
- Supabase未設定 (URL/KEY未設定) 時も安全側: ログイン画面以外は `/login` へ。

1.5 GET /api/me

- 認証必須。応答: `{ id, displayName, role, canEdit(算出), privileged(算出), preferredLocaleId }`。
- 画面はこの値で編集UIの表示可否を切り替える (実権限はサーバで別途強制)。

1.6 PUT /api/me/locale

- 認証必須。Request: `{ localeId: string | null }`。
- `localeId` が非nullなら、有効な `Locale` に存在すること (なければ 404 "ロケールが見つかりません")。
- `users.preferredLocaleId` を更新。応答 `{ ok: true }`。

1.7 GET /api/health

- 認証不要。`{ ok: true, domainVersion, db: { ok, systemSettings(件数) } }`。DB接続不可時は 503 で `db.ok=false`。

2. 物質マスタ

2.1 データ項目

項目	型	制約
code	string	必須・≤100・全物質で一意（正規化）
casNumber	string	必須・≤50・非一意（重複可）
mainName{nameJa, nameEn?}		nameJa 必須・≤500、nameEn ≤500
subNames[] {nameJa, nameEn?}		各 nameJa 必須・≤500、最大100件。表示順=配列順

2.2 GET /api/substances（一覧・検索）

- 認証必須。Query: q? (≤200), page?, pageSize?。
- 検索条件（q があるとき、OR）：codeNormalized に normalize(q) を含む／casNormalized に含む／メイン・サブ名称の nameJa に q を含む／nameEn に q を含む（大文字小文字無視）。
- 並び順: codeNormalized 昇順。
- 応答 items 各要素: 物質 + names（nameType昇順→displayOrder昇順）。{ items, total, page, pageSize }。

2.3 POST /api/substances（登録）

- requireEditor。Request: substanceCreateSchema（2.1）。
- 検証エラーは validation_error 400。
- 処理: 1. codeNormalized=normalize(code), casNormalized=normalize(casNumber)。2. 同一 casNormalized の有効物質件数 sameCasCount を数える。3. Substance を作成。names に MAIN 1件 + SUB（配列順で displayOrder=1..）を同時作成。createdBy=updatedBy=ユーザーID。
- 一意衝突（codeNormalized）→ duplicate_code 409 "物質コード「{code}」は既に使用されています"。
- 応答 201: { item, warnings }。sameCasCount>0 のとき warnings に "同一CAS番号（{cas}）の物質が既に {n} 件登録されています"。

2.4 GET /api/substances/{id}（詳細）

- 認証必須。存在しなければ 404 "物質が見つかりません"。応答 { item }（names + properties含む）。

2.5 PUT /api/substances/{id}（更新）

- requireEditor。Request: substanceUpdateSchema（全項目 optional）。存在チェック→404。
- 処理（トランザクション）：1. 送られた項目のみ更新（code更新時は codeNormalized も、cas更新時は casNormalized も再計算）。updatedBy 更新。2. mainName があれば MAIN 行を更新（nameJa/nameEn）。3. subNames があれば SUB を全削除→再作成（displayOrder=1..）。
- 一意衝突→ duplicate_code 409 "物質コードが既に使用されています"。応答 { item }。

2.6 DELETE /api/substances/{id}（論理削除）

- requireEditor。存在チェック→404。
- 参照件数: compositionLines = この物質を使う組成行数。
- ?force=true でなく参照>0 → referenced 409 "この物質は {n} 件の組成から参照されています。削除する場合は force=true を指定してください" (details.referencedBy)。
- それ以外は deletedAt=now をセット。応答 { ok:true, referencedBy }。

2.7 GET /api/substances/{id}/products（逆引き・この物質を含む製品）

- 認証必須。存在チェック→404。
- ExpandedComposition から substanceId=id かつ製品が有効の行を取得。非特権ユーザーは product.privateFlag=false に限定。並び=製品コード昇順、最大500件。
- 各行: { productId, code, nameJa, contentPct, calculatedAt }。組成非開示 (compositionPublicFlag=false) かつ非特権のとき contentPct=null (画面は「(非開示)」表示)。

3. 金属換算係数マスタ

3.1 項目・一意

項目	制約
casNumber	必須・≤50
metalElement	必須・ <code>^[A-Z][a-z]{0,2}\$</code> (例 Pb)
ratioPct	必須・ $0 < x \leq 100$ ・小数6桁

- 一意キー: (casNormalized, metalElement)。

3.2 GET /api/metal-factors

- 認証必須。Query q?, page?, pageSize?。検索: casNormalized に normalize(q) を含む／metalElement に q を含む。並び=casNormalized,metalElement 昇順。応答 { items, total, page, pageSize } (ratioPctは文字列)。

3.3 POST /api/metal-factors

- requireEditor。Request: metalFactorCreateSchema。
- 処理: 1. casNormalized 算出。マスタに同CASの有効物質が0件なら warnings に "CAS番号「{cas}」は物質マスタに未登録です"。2. (casNormalized, metalElement) の既存を検索。

- 既存が有効→ duplicate 409 "CAS「{cas}」×元素「{element}」の係数は既に登録されています"。
- 既存が論理削除済み→ 復活（casNumber/ratioPct更新・deletedAt=null）。
- 無し→新規作成。
- 応答 201 { item, warnings }。

3.4 PUT /api/metal-factors/{id}

- requireEditor。有効な対象を検索→404 "金属換算係数が見つかりません"。送信項目のみ更新（cas更新時 casNormalized再計算）。一意衝突→409 "同じCAS×元素の係数が既に存在します"。

3.5 DELETE /api/metal-factors/{id}

- requireEditor。→404 or 論理削除（参照チェックなし）。応答 {ok:true}。

4. 製品／原材料マスタ

4.1 項目

項目	型	制約・既定
code	string	必須・≤100・一意
nameJa	string	必須・≤500
nameEn	string?	≤500
usableAsMaterial	bool	既定 false
privateFlag	bool	既定 false。設定は特権のみ
compositionPublicFlag	bool	既定 false。設定は特権のみ

4.2 GET /api/products（一覧・検索）

- 認証必須。Query: q?, page?, pageSize?, usableAsMaterial? ("true"/"false"), excludeId?。
- 絞り込み:
 - 非特権ユーザーは privateFlag=false に限定（非公開製品を一覧から除外）。
 - usableAsMaterial 指定時はその値で絞る。excludeId 指定時はそのIDを除外。
 - q があるとき OR: codeNormalized 含む／ nameJa 含む／ nameEn 含む（大小無視）。
- 並び=codeNormalized昇順。

4.3 POST /api/products（登録）

- requireEditor。Request: `productCreateSchema`。
- フラグ制御: `privateFlag` か `compositionPublicFlag` を送信し、かつ**非特権**なら → `forbidden` 403 "公開フラグの設定はシステム管理者・特権ユーザーのみ可能です"。
- 作成（フラグ未指定は既定値）。一意衝突→409 "製品コード「`{code}`」は既に使用されています"。応答 201 `{item}`。

4.4 GET /api/products/{id}（詳細）

- 認証必須。可視判定: 対象が存在し、かつ「`privateFlag=false` または 特権」なら可視。非可視→404 "製品が見つかりません"（存在を秘匿）。
- `canViewComposition` = 特権 または `compositionPublicFlag`。
- 応答 `{ item: { ...product, compositionLines, compositionMasked } }`。
 - `canViewComposition=false` のとき `compositionLines=null`, `compositionMasked=true`。
 - `true` のとき `compositionLines` = 組成行（作成順）: `{ id, substanceId, childProductId, contentPct(文字列|null), isBalance, substance{code, nameJa}|null, childProduct{code, nameJa}|null }`。

4.5 PUT /api/products/{id}（更新）

- requireEditor。可視判定と同様に非可視→404。Request: `productUpdateSchema`。
- フラグ送信×非特権→403（4.3と同じ）。
- 警告: `usableAsMaterial` を `false` に変更し、かつ原材料参照>0 → `warnings` "この製品は {n} 件の組成から原材料として参照されています（参照は維持されます）"（保存は継続）。
- 送信項目のみ更新。一意衝突→409。応答 `{ item, warnings }`。

4.6 DELETE /api/products/{id}

- requireEditor。非可視→404。参照件数 `compositionLines` = この製品を原材料として使う組成行。参照>0 かつ `force`→409 "この製品は {n} 件の組成から原材料として参照されています。削除する場合は `force=true` を指定してください"。それ以外は論理削除。

5. 原組成（CompositionLine）

5.1 行の構造・検証 (compositionLineSchema)

項目	制約
substanceld? / childProductId?	どちらか一方のみ。両方または両方空→"構成要素は物質または原材料のどちらか一方を指定してください"
contentPct?	単一値・ $0 < x \leq 100$ ・小数6桁。非balance行で未指定→"含有率は必須です (balance行を除く)"
isBalance	既定 false。true かつ contentPct 指定→"balance行に含有率は指定できません"

- lines は最大1000件。

5.2 PUT /api/products/{id}/composition (原組成の全置換)

- requireEditor。Request: { lines: CompositionLineInput[] }。
- 前提チェック: 1. 製品が有効かつ可視 (非公開×非特権→404)。2. 組成編集可否 = 特権 または compositionPublicFlag。不可→ forbidden 403 "この製品の組成は非開示のため編集できません"。
- 業務検証 (エラーが1件でもあれば保存しない。全件を details.errors に格納し 400 composition_invalid "組成の内容に誤りがあります"): 1. **物質の重複**: 同一 substanceld が複数行→"同一の物質が複数行に指定されています"。2. **原材料の重複**: 同一 childProductId が複数行→"同一の原材料が複数行に指定されています"。3. **物質存在**: substanceld が有効物質にない→"物質が見つかりません: {id}"。4. **原材料存在・可否**: childProductId が「有効かつ (非特権なら非公開でない)」でない→"原材料が見つかりません: {id}"。存在するが usableAsMaterial=false →"「{code}」は原材料利用可フラグがオフのため組成に使用できません"。5. **循環参照** (下記5.3) →"循環参照が発生するため登録できません (この製品自身に行き着く原材料が含まれています)"。6. **合計検証** (下記5.4。errors を追加、warnings は保存継続)。
- 保存: 既存行を全削除→新規作成 (balance行は contentPct=null)。product.updatedBy 更新。
- 応答 { ok:true, warnings, totalPct, balancePct }。

5.3 循環参照検出 (wouldCreateCycle)

- 直接の子製品IDリストに親製品ID自身が含まれる→循環 (自己参照)。
- 子製品を起点にBFSで組成グラフを辿り、親製品IDに到達したら循環。visited管理で無限ループを防止。

5.4 含有率合計検証 (validateCompositionSum・システム設定駆動)

入力: 行 (contentPct|null, isBalance) とシステム設定 { mode, epsilonPct, balanceAllowed }。

1. balance行の検証: - balance行あり かつ balanceAllowed=false →エラー"balance (残部) 行はシステム設定で許可されていません"。 - balance行が2件以上→"balance (残部) 行は1製品につき1件までです"。 - balance行に contentPct あり→"balance (残部) 行に含有率は指定できません (自動補完されます)"。 - 非balance行で contentPct なし→"含有率が未入力の行があります"。
2. 既知合計 total = Σ (非balance行の contentPct)。

3. **balanceあり**（許可時）： $\text{balancePct} = 100 - \text{total}$ 。 - $\text{balancePct} < -\varepsilon$ （既知が $100+\varepsilon$ を超過） $\rightarrow$ mode=LENIENT なら警告 "含有率合計が100%を超えています（既知成分合計 {total}%）" + $\text{balance}=0$ 、それ以外はエラー "...balanceが負になります"。 - $-\varepsilon \leq \text{balancePct} < 0 \rightarrow \text{balance}=0$ に丸め。
4. **balanceなし**： - $\text{total} > 100+\varepsilon \rightarrow$ LENIENT は警告、他はエラー "含有率合計が100%を超えています（合計 {total}%）"。 - $\text{total} < 100-\varepsilon \rightarrow$ STRICT はエラー "含有率合計が100%になっていません（合計 {total}%）"、STANDARD は警告 "含有率合計が100%未満です（合計 {total}%・未開示/残部あり）"、LENIENT は何もしない。
5. 戻り値 { errors, warnings, totalPct, balancePct }。
- ※ mode/ ε /balanceAllowed は SystemSetting（10章）から取得。lines が空のときは errors なし・warnings=["組成が空です"]。

6. 組成の展開・統合

6.1 POST /api/products/{id}/expand

- 認証必須。可視判定 $\rightarrow$ 非可視404。**組成閲覧権限**（特権 または compositionPublicFlag）がなければ 403 "この製品の組成は非開示です"。
- 手順: 1. 対象製品を起点に、組成グラフをBFSで全ロード（到達する全製品の CompositionLine）。2. 展開エンジン（6.2）を実行。3. ExpandedComposition（統合後）と ExpandedPath（経路明細）を全置換保存（product単位でdelete $\rightarrow$ createMany、精度12桁）。
- 応答 { calculatedAt, warnings, items[] }。items 各: { substanceId, code, casNumber, nameJa, contentPct(12桁) }。

6.2 展開エンジン（expandComposition・純粹関数・決定的）

入力: rootProductId, compositions (productId $\rightarrow$ 行[]), maxDepth(既定50)。

- 再帰 walk(productId, multiplier, chain, visiting, depth): 1. depth>maxDepth $\rightarrow$ 警告 max-depth-exceeded を出し打ち切り。2. 当該製品の行がない $\rightarrow$ 警告 composition-missing: product={id}、寄与なし。3. balance補完: known= Σ (非balance行pct)、balancePct= $100-\text{known}$ （負なら0に丸め + 警告 negative-balance-clamped）。4. 各行の実効pct = balance行なら balancePct、通常は contentPct。
 - 物質行 $\rightarrow$ paths に { substanceId, contributionPct = multiplier $\times$ pct, path=chain+[{productId, linePct}] } を追加。
 - 製品行 $\rightarrow$ visiting に既出なら警告 cycle-detected でスキップ なければ walk(child, multiplier $\times$ pct/100, ..., depth+1)。
- 統合: paths を substanceId でグルーピングし contributionPct を合算 $\rightarrow$ merged[]（substanceId昇順）。
- 戻り値 { paths, merged, warnings(ソート済) }。
- 計算規則**: 寄与率 = 親含有率 $\times$ 子含有率 $\div 100$ （多段は乗算の連鎖）。同一物質は合算。Decimalで計算し順序非依存。

7. 規制該当判定

7.1 POST /api/products/{id}/judge

- 認証必須（編集権限は不要）。可視判定→非可視404。Request: { `versionId`: string } (`judgeRequestSchema`。未指定→`validation_error`)。
- `version` が存在しない→404 "リンクバージョンが見つかりません"。status=RETIRED なら warnings "廃止済みのバージョン ({code}) で判定を実行しています"。
- 手順: 1. 展開を再実行 (6.2。常に最新組成で判定)。warnings を引き継ぐ。2. 入力データをロード: 展開後物質、全規制区分 (有効・法律有効)、全法文物質名 (有効)、当該version の全リンク、全情報源、全金属換算係数 (有効)。3. 判定エンジン入力を構築 (すべて値・DB非依存。物質は `casNormalized` を持たせる)。4. 判定エンジン (7.2) を実行。5. 結果を保存: `DeterminationRun` (product, version, executedBy, executedAt, `sourcePrioritySnapshot` =情報源優先度の配列) + 区分ごとに `DeterminationResult` (law/category/該当/算定値/採用情報源/isFinal/suppressed) + `DeterminationContribution` (寄与明細)。6. 監査ログ action=determine。
- 応答 { `run{id,executedAt,versionCode}`, `maskedDetails`, warnings, `finalCategories[]`, `results[]` }。
 - `maskedDetails` = 非特権 かつ 組成非開示。true のとき各 result の `contributions=[]` (結論・算定値は返す)。
 - results 各: { `categoryId`, `lawNameJa`, `lawAbbrev`, `categoryNameJa`, `applicable`, `calculatedValue`(6桁|null), `isFinal`, `suppressed`, `contributions[]` }。
 - contribution 各: { `substanceCode`, `substanceNameJa`, `casNumber`, `value`(6桁), `sourceName`, `statutoryNameJa`, `controlNumber` }。

7.2 判定エンジン (judge・純粋関数・決定的)

入力を `substanceId` 昇順・`category id` 昇順・`source` は `priority`昇順→`id`昇順 (active のみ) に整列してから処理する。オプション既定: 下限・上限とも境界を含む (以上・以下)。

A. 有効閾値 (法文物質名 `s` と区分 `c`) : `s` に `thresholdLower/Upper` のいずれかがあれば `s` の値、なければ `c` の値。両方nullなら「判定不能」(該当量が閾値未設定→警告 `threshold-missing`、該当としない)。

B. 閾値判定 `withinThreshold(value, th)`:

- `th` 未設定→false。
- 下限あり: `lowerInclusive` ? `value`≥`lower` : `value`>`lower`、満たさなければ false。
- 上限あり: `upperInclusive` ? `value`≤`upper` : `value`<`upper`、満たさなければ false (上限超過は非該当)。

C. 情報源フォールバック `resolveAdoptedLinks`(法文物質名, `links`, `sortedSources`):

- 優先度順に情報源を見て、その法文物質名×その情報源のリンクCAS集合が1件以上ある最初の情報源を採用。採用情報源のCAS集合を返す。なければ null。

D. 寄与量 `contribution`(物質`e`, 法文物質名`s`):

- `s.judgmentMethod=METAL_CONVERSION` かつ `metalElement` あり: 係数 f =金属換算係数 (`casNormalized=e.cas` かつ `metalElement`)。無ければ警告 `metal-factor-missing` + 寄与0。あれば $e.contentPct \times f.ratioPct \div 100$ 。
- それ以外: `e.contentPct`。

E. 区分の独立評価 `evaluateIndependent(c)` (兼ね合いOFF) — 合算方式で分岐:

- **STATUTORY_NAME**: 各法文物質名 `s` で採用リンク解決→組成の該当物質を集め、`amount=Σ contribution(i,s)`。有効閾値で判定。該当なら区分該当・寄与明細を記録。区分の算定値は該当名称の最大該当量 (非該当時は観測最大量)。
- **CATEGORY_TOTAL**: 全法文物質名の該当物質を`distinct (1物質1回)`でプールし、`amount=Σ contentPct` (含有率方式前提。金属換算が混ざる場合は警告 `metal-conversion-in-category-total` を出しつつ含有率で計上)。区分閾値で判定。
- **NONE**: 物質ごとに個別に `contribution` を閾値判定。1つでも該当で区分該当。

F. 兼ね合い評価 `evaluateAccumulated(c)` (兼ね合いON) :

- `c.rank` が null→警告 `rank-missing`、非該当。
- 同一 `interactionGroup` 内で $rank \leq c.rank$ の区分 (自区分含む = 上位 + 自区分) の法文物質名の該当物質をプール (同一物質は1回、寄与は最大値を採用)。
- プール合計を **区分閾値** と比較 (合算方式によらずプール合算。Q-G8暫定)。該当なら寄与明細を記録。

G. 結果抑制 (Q-J4) :

- 兼ね合いグループごとに、該当区分のうち **rank最小** を `isFinal=true`。他の該当区分は、その区分の `suppressResultFlag` が真なら `suppressed=true`、偽なら `isFinal=true`。
- グループに属さない区分は、該当なら `isFinal=true`。
- rank重複がグループ内にあれば警告 `rank-duplicated`。
- `finalCategories` = `isFinal` の `categoryId` を昇順。

必須テスト (別紙 `judgment-engine.md T1~T13`) : 特に T5 = 3種閾値5%・3種物質3% + 2種物質3%→累積6%≥5%で3種該当・最終=3種のみ。

7.3 判定履歴

- **GET /api/products/{id}/runs**: 認証必須・非可視404。最新20件。各 { `id`, `executedAt`, `versionCode`, `finalCategories[]` ("略称/区分名") }。
- **GET /api/runs/{id}**: 保存済み結果の再表示。非可視404。`maskedDetails` は判定APIと同一方針。応答形は7.1と同形 (履歴の `contribution` には `statutoryNameJa/controlNumber` は空)。

8. 法規制マスタ

8.1 法律 (Law)

項目: code(必須≤100一意)/nameJa(必須≤500)/nameEn?(≤500)/abbrev?(≤100)/note?(≤2000)。

- GET /api/laws: 認証。検索 OR: codeNormalized/nameJa/nameEn/abbrev。並び=codeNormalized。
- POST /api/laws: requireEditor。一意衝突→409 "法律コード「{code}」は既に使用されています"。
- GET /api/laws/{id}: 認証。詳細 + 配下区分一覧（各区分に statutorySubstanceCount）。→404 "法律が見つかりません"。
- PUT /api/laws/{id}: requireEditor。送信項目のみ更新。
- DELETE /api/laws/{id}: requireEditor。配下有効区分>0 かつ非force→409 "この法律には {n} 件の規制区分があります。削除する場合は force=true を指定してください（区分・法文物質名も参照不可になります）"。

8.2 規制区分 (RegulationCategory)

項目: code(必須≤100・法律内一意)/nameJa(必須)/nameEn?/rank?(整数≥1)/interactionFlag(既定false)/interactionGroup?(≤100)/summationMethod(必須・STATUTORY_NAME|CATEGORY_TOTAL|NONE)/thresholdLower?・thresholdUpper?(0≤x≤100)/suppressResultFlag(既定true)。lawId は作成時のみ・変更不可。

- 業務ルール（validateCategoryRules。作成は入力に、更新はマージ後状態に適用。違反は validation_error 400・details.errors）：
 - interactionFlag=true かつ rank=null→"兼ね合いフラグがオンの区分には rank（順位）が必須です"。
 - interactionFlag=true かつ interactionGroup空→"兼ね合いフラグがオンの区分には兼ね合いグループが必須です"。
 - thresholdLower>thresholdUpper→"閾値の下限が上限を超えています"。
- POST /api/categories: requireEditor。lawId の法律が有効であること（→404）。一意衝突→409 "区分コード「{code}」はこの法律内で既に使用されています"。
- GET /api/categories/{id}: 認証。詳細 + law + 法文物質名一覧（controlNumber昇順）。
- PUT /api/categories/{id}: requireEditor。マージ後にルール検証。一意衝突→409。
- DELETE /api/categories/{id}: requireEditor。配下法文物質名>0 かつ非force→409 "この区分には {n} 件の法文物質名があります。..."。

8.3 法文物質名 (StatutorySubstance)

項目: controlNumber(必須≤100・区分内一意)/nameJa(必須)/nameEn?/thresholdLower?・thresholdUpper?/judgmentMethod(必須・CONTENT_RATIO|METAL_CONVERSION)/metalElement?(^[A-Z][a-z]{0,2}\$)。

- 業務ルール（validateStatutoryRules。更新はマージ後）：
 - METAL_CONVERSION かつ metalElement空→"金属換算方式の法文物質名には判定金属元素が必須です"。

- CONTENT_RATIO かつ metalElement あり→"含有率方式の法文物質名に判定金属元素は指定できません"。
- 下限>上限→"該当含有率範囲の下限が上限を超えています"。
- GET /api/statutory-substances?q=: 認証。横断検索 (nameJa/nameEn/controlNumber、最大 pageSize=10既定)。各 { id, controlNumber, nameJa, category{ id, nameJa, law{id, nameJa, abbrev} } } (リンク登録のピッカー用)。
- POST /api/statutory-substances: requireEditor。categoryId 有効→404。一意衝突→409 "管理番号 [{no}]はこの区分内で既に使用されています"。
- PUT /api/statutory-substances/{id}: requireEditor。マージ後ルール検証。
- DELETE /api/statutory-substances/{id}: requireEditor。リンク参照>0 かつ非force→409 "この法文物質名は {n} 件のCASリンクから参照されています..."。

9. リンク管理

9.1 リンクバージョン (LinkSetVersion)

項目: versionCode(必須≤50・一意)/name?(≤500)/status?(DRAFT|ACTIVE|RETIRED)。

- GET /api/link-versions: 認証。全件 (versionCode降順) + linkCount, runCount。
- POST /api/link-versions: requireEditor。status未指定は DRAFT、loadedAt=now。一意衝突→409 "バージョンコード[{code}]は既に使用されています"。
- GET /api/link-versions/{id}: 認証。詳細 + linkTotal + perSource [{sourceId, sourceName, priority, count}] (全情報源、priority昇順)。
- PUT /api/link-versions/{id}: requireEditor。送信項目のみ更新。
- DELETE /api/link-versions/{id}: requireEditor (物理削除)。リンク>0 または 判定実行>0→referenced 409 "このバージョンは削除できません (リンク {a} 件・判定実行 {b} 件が存在します)。不要な場合は状態を「廃止」にしてください"。
- POST /api/link-versions/{id}/duplicate: requireEditor。Request { versionCode, name? }。新バージョン (status=DRAFT) を作り、元版の全リンクを新版へコピー。応答 201 { item, copiedLinks }。コード衝突→409。

9.2 情報源 (Source)

項目: name(必須≤100・一意)/priority(整数1..1000)/activeFlag(既定true)。

- GET /api/sources: 認証。priority昇順 + linkCount。
- POST /api/sources: requireEditor。name重複→409 "情報源[{name}]は既に登録されています"。
- PUT /api/sources/{id}: requireEditor。→404。name変更時の重複→409。送信項目のみ更新。

- **DELETE /api/sources/{id}**: requireEditor（物理削除）。リンク参照>0→409 "この情報源は {n} 件のリンクから参照されているため削除できません。使用を止める場合は有効フラグをオフにしてください"。

9.3 リンク（StatutoryCasLink）

項目: versionId/statutorySubstanceId/sourceId/casNumber(必須≤50)。

- **GET /api/links?versionId=**: 認証。必須 versionId。任意 sourceId, statutorySubstanceId, q (casNormalized/法文物質名/管理番号)。並び=casNormalized。各要素に source・statutorySubstance (区分・法律含む) を展開。
- **POST /api/links**: requireEditor。Request { versionId, statutorySubstanceId, sourceId, casNumber }。
 - version/statutory/source の存在チェック（各→404）。
 - 重複（version×statutory×source×casNormalized）→409 "同じリンクが既に登録されています"。
 - 物質マスタに同CASの有効物質が0件→ warnings "CAS番号「{cas}」は物質マスタに未登録です（判定では突合されません）"。
 - 応答 201 { item, warnings }。
- **DELETE /api/links/{id}**: requireEditor（物理削除）。判定根拠（DeterminationContribution.linkId）から参照>0→409 "このリンクは {n} 件の判定根拠から参照されているため削除できません"。

10. ユーザー管理・システム設定（管理者）

10.1 GET /api/users

- requireAdmin。有効ユーザー一覧（createdAt昇順）: { id, displayName, role, canEdit, activeFlag, createdAt }。

10.2 PUT /api/users/{id}

- requireAdmin。Request { displayName?, role?, canEdit?, activeFlag? }。→404。
- **自己ロックアウト防止**（id=自分）: role を SYSTEM_ADMIN 以外に→400 "自分自身のシステム管理者権限は変更できません"。activeFlag=false→400 "自分自身を無効化することはできません"。
- **canEdit の強制**: 結果ロールが SYSTEM_ADMIN/PRIVILEGED なら canEdit は強制 true。NON_PRIVILEGED のときのみ送信値（未指定は現状維持）。
- 送信項目のみ更新。応答 { item }。

10.3 システム設定（SystemSetting）

キーは3つ固定:

キー	型	既定	値検証
composition.validation_mode	string	STANDARD	STRICT/STANDARD/LENIENT のみ（不正→"検証モードは STRICT / STANDARD / LENIENT のいずれかです"）
composition.epsilon	number	0.01	0～10 の数値（不正→"許容誤差 ε は 0～10 の数値で入力してください"）
composition.balance_allowed	boolean	true	"true"/"false" のみ（不正→"balance許容は true / false のいずれかです"）

- **GET /api/settings:** requireAdmin。全件（key昇順）。
- **PUT /api/settings:** requireAdmin。Request { key, value }。key は上記3つのenum。値検証→不正は 400 validation_error（上記メッセージ）。→存在しないkeyは404。更新し updatedBy 記録。
- 設定値の読み出しは getCompositionSettings(): 各キーを value ?? defaultValue ?? ハードコード既定 の順で解決。組成保存・TSV取込の合計検証（5.4）に適用。

11. 多言語（管理者）

11.1 ロケール（Locale）

項目: code(^[a-z]{2}(-[A-Za-z0-9]{2,8})?\$.一意)／displayName(必須≤100)／isDefault(1件のみ)／activeFlag(既定true)。

- **GET /api/locales:** 認証（activeFlag=true のみ、code昇順）。?all=true は requireAdmin（無効含む全件）。
- **POST /api/locales:** requireAdmin。code重複→409 "ロケール「{code}」は既に登録されています"。作成時 activeFlag=true。
- **PUT /api/locales/{id}:** requireAdmin。Request { displayName?, activeFlag?, isDefault? }。
 - 既定ロケールの無効化/既定解除は禁止→400 "既定ロケールは無効化できません。先に別のロケールを既定にしてください"。
 - isDefault=true にするとき、他の isDefault を全解除してから設定（常に1件）。

11.2 UI翻訳（UiTranslation）

- **GET /api/translations?localeId=:** requireAdmin。当該ロケールの { id, key, value }（key昇順）。
- **PUT /api/translations:** requireAdmin。Request { localeId, key(≤200), value(≤2000) }。(localeId, key) で upsert。ロケール存在チェック→404。
- **DELETE /api/translations?id=:** requireAdmin。→404 or 削除。
- **GET /api/i18n/catalog?locale=:** 認証。既定ロケールの全翻訳をベースに、指定ロケールの値で上書き（未翻訳キーは既定ヘッフォールバック）。応答 { locale, catalog: {key:value} }。
- クライアントは t(key, fallback) でカタログを引き、無ければ fallback（コード内の日本語）を表示。

12. TSV 取込／出力

12.1 形式（共通）

- UTF-8・BOM付き。タブ区切り。1行目ヘッダ。改行 CRLF。値内のタブ・改行は空白へ置換。
- ダウンロードは Content-Disposition: attachment; filename*=UTF-8' '<encoded>。
- 取込は Content-Type: text/tab-separated-values、本文にTSV文字列。上限 10,000行。エラーが1件でもあれば全件ロールバック（何も取り込まない）。

12.2 出力 API（GET・認証必須。`?template=true` はヘッダのみ）

- `/api/export/substances`: 列 `code`, `cas_number`, `name_ja`, `name_en`, `sub_names_ja`, `sub_names_en`（サブは | 区切り、和英は位置対応）。
- `/api/export/products`: 列 `code`, `name_ja`, `name_en`, `usable_as_material`, `private_flag`, `composition_public_flag`。非特権は非公開製品を除外。
- `/api/export/laws`: 法文物質名1件=1行に階層を横展開（列は第11章 TSV定義参照）。
- `/api/export/links?versionId=`: 列 `law_code`, `category_code`, `control_number`, `cas_number`, `source_name`。`versionId`必須（無→400）、版なし→404。
- `/api/export/metal-factors`: 列 `cas_number`, `metal_element`, `ratio_pct`。

12.3 取込 API（POST・requireEditor）

共通: `parseTsv` でBOM除去・空行スキップ・必須列チェック（不足→"必須列「{col}」がヘッダにありません"）。データ0行→"データ行がありません"。

- **POST `/api/import/substances`**（必須列 `code,cas_number,name_ja`）：
 - 行検証: `code/cas/name_ja` 空→"{行}行目: {列} が空です"。ファイル内 `code`重複→"{行}行目: `code` 「{code}」がファイル内で重複しています"。
 - `upsert`: `codeNormalized` で既存検索。既存→更新（コード/CAS/CAS正規化・MAIN名称更新・SUB全置換）、無→新規（MAIN + SUB作成）。サブ名称は `sub_names_ja` | `sub_names_en` を位置対応。
 - 応答 { `ok`, `created`, `updated`, `total` }。
- **POST `/api/import/metal-factors`**（必須列 `cas_number,metal_element,ratio_pct`）：
 - 行検証: `cas`空／`element`形式不正／`ratio`範囲外→行番号付メッセージ。ファイル内 `CAS`×元素重複→エラー。
 - `upsert`: (`casNormalized`, `metalElement`)。既存→更新（`deletedAt=null`復活含む）、無→新規。
- **POST `/api/import/links?versionId=`**（必須列 `law_code,category_code,control_number,cas_number,source_name`）：
 - `versionId`必須（無→400）・版存在（無→404）。

- 名前解決: law_code→法律、(law,category_code)→区分、(category,control_number)→法文物質名、source_name→情報源。解決不能は各行 "...が見つかりません"。
- 既存と同一 (statutory×source×casNormalized) はスキップ、新規のみ createMany。
- 応答 { ok, created, skippedExisting, total }。

13. ドキュメント生成

13.1 テンプレート (DocumentTemplate)

- **POST /api/doc-templates** (multipart。特権のみ、非特権→403 "テンプレートの登録は...") : file(.docx・≤10MB・拡張子 .docx 以外→400 "v1 では Word (.docx) テンプレートのみ対応しています (Excel は今後対応) ")、name (必須)、outputFormat (SAME/PDF/BOTH)。実体は baseFile (Bytes) にDB格納。targetTypes=["PRODUCT","JUDGMENT"]。
- **GET /api/doc-templates**: 認証。一覧 { id, name, baseFileRef, baseFormat, outputFormat, createdAt }。
- **DELETE /api/doc-templates/{id}**: 特権のみ→論理削除。

13.2 POST /api/doc-templates/{id}/generate

- 認証必須 (生成は誰でも可)。Request { productId, format? }。テンプレート/製品の存在・可視チェック (非可視→404)。
- 差込データ (呼出ユーザーの権限でマスキング) :
 - product{code,name_ja,name_en}, generated_at (日本語日時), version_code (最新判定のバージョン), masked (組成閲覧可なら空、不可なら" (組成非開示) ")。
 - composition[] (組成閲覧可のときのみ。ExpandedComposition から {code,cas,name,pct})。不可なら空配列。
 - results[] (最新 DeterminationRun の結果から {law(略称),category,applicable(該当/非該当),value,final(最終該当/抑制/空)})。
- レンダリング: docxtemplater (paragraphLoop:true, linebreaks:true)。タグ不正→400 "テンプレートの差込に失敗しました (タグの記法を確認してください)"。
- PDF変換 (format=PDF または 既定がPDF) : LibreOffice(soffice)を既定パスから検出。無→501 "PDF変換には LibreOffice のインストールが必要です (Word形式では生成できます)"。ありならヘッドレス変換。
- 生成履歴 (GeneratedDocument) をメタのみ保存 (fileRef=null)。監査 action=export。
- 応答: バイナリ (docx or pdf) を Content-Disposition: attachment; filename*=UTF-8'<製品コード_テンプレ名>.<ext>。

13.3 差込タグ仕様（Q-DOC1）

タグ	展開内容
<code>{product.code}</code> <code>{product.name_ja}</code> <code>{product.name_en}</code>	製品情報
<code>{generated_at}</code> <code>{version_code}</code> <code>{masked}</code>	生成日時・使用版・非開示表示
<code>{#composition}...{/composition}</code>	内側 <code>{code}{cas}{name}{pct}</code> 。表の行に置くと件数分の行に展開
<code>{#results}...{/results}</code>	内側 <code>{law}{category}{applicable}{value}{final}</code>

14. 列挙値・既定値まとめ（再現用）

Enum	値
Role	SYSTEM_ADMIN, PRIVILEGED, NON_PRIVILEGED
SubstanceNameType	MAIN, SUB
SummationMethod	STATUTORY_NAME, CATEGORY_TOTAL, NONE
JudgmentMethod	CONTENT_RATIO, METAL_CONVERSION
VersionStatus	DRAFT, ACTIVE, RETIRED
TemplateOutputFormat	SAME, PDF, BOTH
CompositionValidationMode	STRICT, STANDARD, LENIENT

初期シード: 情報源 USER(1)/LOLI(2)/CHRIP(3)/JAMP(4)、ロケール ja(既定)/en、SystemSetting 3件（上記既定）、SubstancePropertyDef（smiles/inchikey/molecular_formula/melting_point/density）。

別紙（詳細は各文書）

- 判定エンジンのテストケース詳細: judgment-engine.md
- テーブル定義・制約・インデックス・移植性: data-model.md
- 現行画面のUI詳細（ボタン配置・文言）: 機能仕様書.md

改訂履歴

版	日付	内容
1.0	2026-07-03	初版（API入出力・検証・アルゴリズムを再現可能な粒度で記述）